CHỦ ĐỀ 4 · MÔN LẬP TRÌNH ĐA NỀN TẢNG

Kiến trúc ứng dụng Flutter: Mô hình MVVM & Quản lý trạng thái tách biệt

Ứng dụng demo: Posts Reader, đọc bài viết từ REST API

Thành viên **Đặng Công Thắng** · Lớp 23KTMT2 · MSSV: 106230061

Võ Văn Thái · Lớp 23KTMT1 · MSSV: 106230027

Giảng viên Nguyễn Duy Nhật Viễn

Trường Đại học Bách khoa – Đại học Đà Nẵng · Khoa Điện tử Viễn thông

Mã nguồn công khai: `github.com/cityengin/posts-reader`

Phân công

Đặng Công Thắng	Võ Văn Thái
• Business Logic • State Management • Slide	• UI Design • API
52%	**48%**

MVVM: ba thành phần với ranh giới rõ ràng

Tách ứng dụng thành ba phần để giao diện không chứa logic: đổi giao diện không đụng logic, và logic kiểm thử được mà không cần dựng giao diện.

Model

Thư mục:
`models/post.dart`

Vai trò:
Dữ liệu và cách phân tích JSON

ViewModel

Thư mục:
`viewmodels/`

Vai trò:
Logic nghiệp vụ, giữ trạng thái

View

Thư mục:
`views/`

Vai trò:
Giao diện, chỉ hiển thị

Truy cập mạng không nằm trong ViewModel mà tách thành **Network Service** (`services/`): ViewModel gọi service, service trả về Model hoặc ném lỗi đã phân loại.

MVVM trong code: Model, ViewModel, View

models/post.dart (Model)

```dart
// Nơi DUY NHẤT giải mã JSON thành Post.
factory Post.fromJson(Map<String, dynamic> json) {
  return Post(
    id: json['id'] as int,
    userId: json['userId'] as int,
    title: json['title'] as String,
    body: json['body'] as String,
  );
}
```

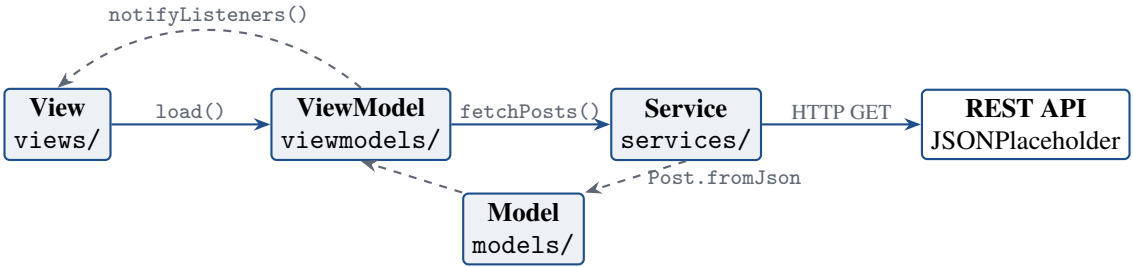
post_list_view_model.dart (ViewModel)

```dart
class PostListViewModel extends BaseViewModel {
  PostListViewModel(this._service); // nhận service
  final PostService _service;
  List<Post> _posts = const [];
  List<Post> get posts => _posts;   // View chỉ đọc
  // Gọi khi mở màn hình và khi bấm "Thử lại".
  Future<void> load() => runLoad(() async {
    _posts = List.unmodifiable(
        await _service.fetchPosts());
  });
}
```

views/post_list_view.dart (View)

```dart
// View chỉ lắng nghe ViewModel và vẽ theo trạng thái.
// Không có http, không có JSON.
ListenableBuilder(
  listenable: _viewModel,
  builder: (context, _) {
    return switch (_viewModel.status) {
      // Đang tải: vòng xoay.
      LoadStatus.loading => const Center(
        child: CircularProgressIndicator(),
      ),
      // Lỗi: hiện thông báo; nút "Thử lại" chỉ có
      // khi ViewModel cho phép (canRetry).
      LoadStatus.error => ErrorRetryView(
        message: _viewModel.errorMessage ?? '',
        onRetry: _viewModel.canRetry
            ? _viewModel.load
            : null,
      ),
      // Thành công: vẽ danh sách lấy từ ViewModel.
      LoadStatus.success =>
        _buildList(context, _viewModel.posts),
    };
  },
)
```

Kiến trúc ứng dụng



`lib/main.dart`	Lắp ráp: tạo `PostService`, truyền hàm tạo ViewModel cho View
`lib/models/`	`post.dart`
`lib/services/`	`post_service.dart`, `api_exception.dart`
`lib/viewmodels/`	`base_view_model.dart`, `post_list_view_model.dart`, `post_detail_view_model.dart`
`lib/views/`	`post_list_view.dart`, `post_detail_view.dart`, `widgets/error_retry_view.dart`

Lắp ráp các thành phần trong main.dart

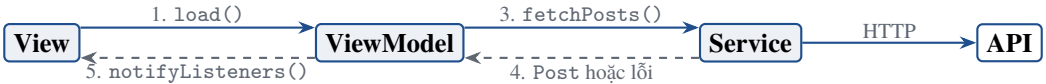
lib/main.dart

```dart
// Nơi DUY NHẤT biết PostService (composition root).
void main() {
  // Một service dùng chung cho cả ứng dụng.
  final service = PostService();

  runApp(
    PostsReaderApp(
      // Hàm tạo ViewModel: View gọi khi cần,
      // View không biết PostService là gì.
      createListViewModel: () =>
          PostListViewModel(service),
      createDetailViewModel: (id) =>
          PostDetailViewModel(service, id),
    ),
  );
}
```

- Chỉ `main.dart` import `services/`.
- View nhận **hàm tạo** ViewModel (factory) thay vì nhận service.
- Mỗi View tự tạo ViewModel trong `initState` và tự hủy trong `dispose`.
- Nhờ vậy test có thể truyền ViewModel dùng service giả.

Luồng dữ liệu một chiều



1. View gọi `load()`.
2. ViewModel đặt `loading` và thông báo.
3. ViewModel nhờ Service lấy dữ liệu.
4. Service trả `Post` hoặc ném `ApiException`.
5. ViewModel đặt `success/error` và thông báo.
6. View vẽ lại theo `status`.

Nét liền: **hành động** đi xuống. Nét đứt: **trạng thái** đi ngược lên. View chỉ **đọc** trạng thái qua getter và không tự đổi trạng thái.

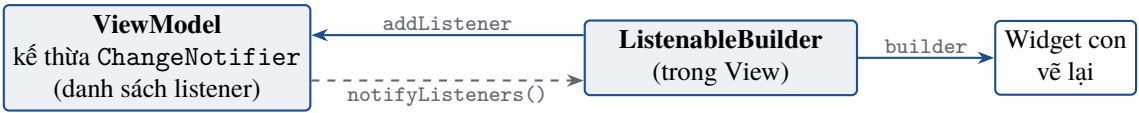
Luồng dữ liệu trong code: runLoad()

viewmodels/base_view_model.dart

```dart
Future<void> runLoad(
    Future<void> Function() action) async {
  // (1) Báo cho View: đang tải.
  _status = LoadStatus.loading;
  _errorMessage = null;
  _canRetry = false;
  notifyListeners();
  try {
    await action();          // (2) gọi service lấy dữ liệu
    _status = LoadStatus.success;
  } on ApiException catch (e) {   // (3) lỗi đã phân loại
    _status = LoadStatus.error;
    _errorMessage = e.message;    // câu báo lỗi tiếng Việt
    _canRetry = e.isRetryable;    // có hiện nút "Thử lại"?
  } catch (_) {                   // (4) lỗi bất ngờ
    _status = LoadStatus.error;
    _errorMessage = 'Đã xảy ra lỗi không xác định.';
    _canRetry = true;
  }
  notifyListeners();         // (5) Báo View: xong, vẽ lại.
}
```

- `load()` của ViewModel con chỉ truyền việc cần làm vào `runLoad`.
- Trạng thái đổi **hai lần**: đang tải, rồi thành công hoặc lỗi.
- Hàm **không ném exception**: lỗi thành trạng thái, View không cần `try-catch`.

ChangeNotifier và ListenableBuilder: mẫu Observer



- **Subject**: ViewModel giữ danh sách người lắng nghe. **Observer**: `ListenableBuilder` đăng ký khi được tạo và hủy đăng ký khi bị gỡ.
- `notifyListeners()` gọi lần lượt từng listener, `ListenableBuilder` chạy lại `builder`. Chỉ phần trong `builder` được vẽ lại, không phải cả màn hình.
- `ChangeNotifier` thuộc thư viện nền (`foundation`), `ListenableBuilder` có sẵn trong Flutter: không cần package quản lý trạng thái nào.

Vòng đời: tạo, lắng nghe và hủy an toàn

viewmodels/base_view_model.dart

```dart
bool _disposed = false;

// Người dùng có thể thoát màn hình khi
// request còn đang chạy. Bỏ qua thông báo
// sau dispose để tránh lỗi dùng
// ChangeNotifier đã hủy.
@override
void notifyListeners() {
  if (!_disposed) {
    super.notifyListeners();
  }
}

@override
void dispose() {
  _disposed = true;  // đánh dấu đã hủy
  super.dispose();
}
```

views/post_list_view.dart (State)

```dart
// "Ai tạo thì người đó dispose":
// State tạo ViewModel, tự hủy khi rời màn.
late final PostListViewModel _viewModel;

@override
void initState() {
  super.initState();
  _viewModel = widget.createViewModel();
  _viewModel.load();  // tải dữ liệu ngay
}

@override
void dispose() {
  _viewModel.dispose();  // hủy ViewModel
  super.dispose();
}
```

Network Service: đóng gói việc gọi REST API

- `PostService` dùng package `http`, không import Flutter.
- Nhận `http.Client` qua constructor: test thay bằng `MockClient`, không cần mạng thật.
- Chỉ có hai hàm: `fetchPosts()` và `fetchPostById(id)`.
- **Mọi** lỗi ném ra ngoài đều là `ApiException` có phân loại.

`Loại lỗi`	**Khi nào**	**Thử lại**
`network`	`ClientException` hoặc quá 10 giây	Có
`server`	HTTP ngoài 2xx, trừ 404	Có
`notFound`	HTTP 404	**Không**
`parse`	JSON sai cú pháp, thiếu field, sai kiểu	Có

Lỗi ngoài danh sách (lỗi lập trình) không bị đổi tên: ViewModel hiện "Đã xảy ra lỗi không xác định."

PostService: gọi API và phân loại lỗi

post_service.dart (_get: phần try)

```
Future<T> _get<T>(
    String path, T Function(Object? json) decode) async {
  try {
    final response = await _client
        .get(Uri.parse('$_baseUrl$path'))
        .timeout(_timeout);          // tối đa 10 giây

    final statusCode = response.statusCode;
    if (statusCode == 404) {         // không có bài viết
      throw const ApiException.notFound();
    }
    if (statusCode < 200 || statusCode > 299) {
      throw ApiException.server(statusCode);
    }

    // Giải mã UTF-8 từ bytes để không hỏng tiếng Việt.
    return decode(
        jsonDecode(utf8.decode(response.bodyBytes)));
  }
  // ... các nhánh catch ở bên phải
```

post_service.dart (_get: phần catch)

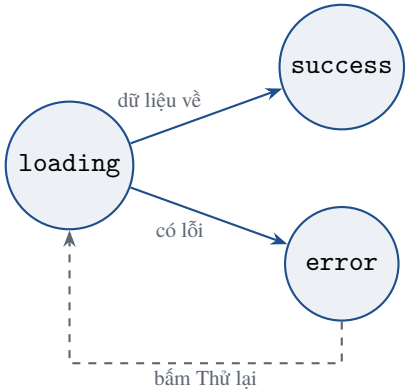
```
  on ApiException {
    rethrow; // đã phân loại, giữ nguyên
  } on http.ClientException {
    throw const ApiException.network(); // mất mạng
  } on TimeoutException {
    throw const ApiException.network(); // quá hạn
  } on FormatException {
    throw const ApiException.parse(); // sai JSON
  } on TypeError {
    throw const ApiException.parse();  // sai field
  }
}
```

services/api_exception.dart

```
enum ApiErrorType { network, server, notFound, parse }

// Bài viết không tồn tại thì thử lại cũng vô ích.
bool get isRetryable =>
    type != ApiErrorType.notFound;
```

Ba trạng thái: Loading, Success, Error và Thử lại



`status`	**View hiển thị**
`loading`	Vòng xoay ở giữa màn hình
`success`	Danh sách, hoặc tiêu đề và nội dung
`error`	Biểu tượng lỗi, thông báo, nút Thử lại nếu `canRetry`

Với lỗi `notFound`, `canRetry = false`: View truyền `onRetry: null` nên không có nút Thử lại, vì thử lại cũng cho kết quả như cũ.

Ba trạng thái trong code

viewmodels/base_view_model.dart

```dart
/// Trạng thái của một lần tải dữ liệu.
enum LoadStatus { loading, success, error }

// Mặc định là loading vì View gọi
// load() ngay khi mở màn hình.
LoadStatus _status = LoadStatus.loading;
String? _errorMessage;
bool _canRetry = false;

// View chỉ đọc qua getter, không tự sửa.
LoadStatus get status => _status;
String? get errorMessage => _errorMessage;
bool get canRetry => _canRetry;
```

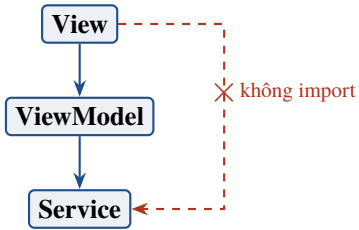
views/widgets/error_retry_view.dart

```dart
class ErrorRetryView extends StatelessWidget {
  const ErrorRetryView({
      super.key, required this.message, this.onRetry});

  final String message;
  // null nghĩa là không cho thử lại.
  final VoidCallback? onRetry;

  // ... trong build(), cột nội dung gồm:
  Icon(Icons.error_outline, size: 48, ...),
  Text(message, textAlign: TextAlign.center, ...),
  // Chỉ hiện nút khi onRetry khác null.
  if (onRetry != null) ...[
    const SizedBox(height: 16),
    FilledButton(
      onPressed: onRetry,
      child: const Text('Thử lại'),
    ),
  ],
}
```

Chứng minh: View không gọi mạng, không giải mã JSON



Mọi import trong `lib/views/` chỉ gồm:

- `package:flutter/material.dart`
- `models/post.dart`
- `viewmodels/*`
- `views/*`

Lệnh kiểm tra, kết quả đúng là **không in ra gì**:

```
> findstr /s /i /m "services/ package:http dart:convert jsonDecode fromJson" lib\views\*.dart
> (không có kết quả)
```

Kiểm thử không cần giao diện, không cần mạng

test/viewmodels/post_list_view_model_test.dart

```dart
// Service giả: test quyết định kết quả, không gọi mạng.
class FakePostService implements PostService {
  Future<List<Post>> Function() onFetchPosts =
      () async => const [];

  @override
  Future<List<Post>> fetchPosts() => onFetchPosts();
  // ... fetchPostById tương tự
}

test('load thành công: loading → success', () async {
  service.onFetchPosts = () async => posts;
  await viewModel.load();

  // Hai lần thông báo: đang tải, rồi thành công.
  expect(notifiedStatuses,
      [LoadStatus.loading, LoadStatus.success]);
  expect(viewModel.posts, posts);
});
```

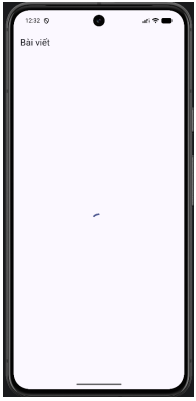
test/services/post_service_test.dart

```dart
test('status 500 → ApiException server', () async {
  final service = PostService(
    // http.Client giả: luôn trả về status 500.
    client: MockClient(
        (_) async => http.Response('', 500)),
  );

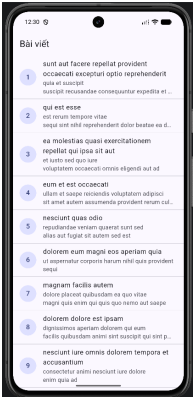
  await expectLater(
    service.fetchPosts(),
    throwsA(isA<ApiException>()
        .having((e) => e.type, 'type',
            ApiErrorType.server)),
  );
});
```

18 test: service 7, ViewModel 6, View 5. `flutter analyze`: không có lỗi.

Demo



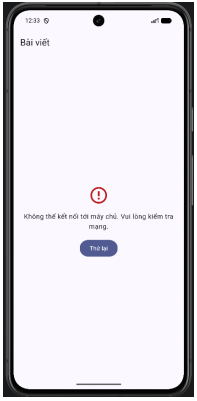
Màn lúc tải



Màn danh sách



Màn chi tiết bài viết



Màn lỗi kèm nút Thử lại

Kịch bản: mở danh sách → bấm vào một bài → tắt mạng và mở lại để thấy lỗi → bật mạng và bấm Thử lại.

Kết luận

- Ứng dụng đọc REST API theo MVVM với bốn thư mục `models/`, `services/`, `viewmodels/`, `views/`.
- Luồng dữ liệu một chiều: View gọi ViewModel, ViewModel đổi trạng thái và thông báo, View vẽ lại.
- Ba trạng thái Loading, Success, Error kèm nút Thử lại; lỗi không thể khắc phục thì không hiện nút.
- View không chứa mã gọi mạng hay giải mã JSON, đã kiểm tra bằng lệnh; 18 test đều đạt.
- Mã nguồn công khai: `github.com/cityengin/posts-reader`.

Tài liệu tham khảo

- Flutter, *Guide to app architecture*: `docs.flutter.dev/app-architecture/guide`
- Flutter API, *ChangeNotifier* và *ListenableBuilder*: `api.flutter.dev`
- Dart team, package *http*: `pub.dev/packages/http`
- JSONPlaceholder, *Free fake REST API*: `jsonplaceholder.typicode.com`